

Shreyas — Speaking Script

Slides owned: 1, 2, 3, 4 (intro half), 8, 10, 13, 14, 16, 19.

Spoken prose, not bullets. ~12 minutes total across all ten slides. Read it through twice before the rehearsal — the goal is to know the *beats* by heart so you can deliver them naturally, not memorise the words.

Slide 1 — Title (~15 seconds)

Sonder.

Beat. Let the wordmark sit on screen.

Travel, together.

Beat.

I'm Shreyas. With me — Mushahid, Jahnvi, and Ali. Over the next half hour we'll walk you through what we built, why we built it, and the parts we're proud of.

Advance.

Slide 2 — The problem (~75 seconds)

Travel software today is fragmented across four broken surfaces, and none of them solve the actual problem.

TripAdvisor gives you ranked lists of restaurants and "things to do" — context-free, generic, and unconnected to whoever you actually are. Google Travel hands you a hotel and a flight and walks away. Instagram inspires you, but a saved reel is not an itinerary. Airbnb books you a room.

So you stitch a trip together across four tabs, copying activity names into a Notes doc, and you arrive at the airport with no plan that knows anything about you.

Then there's the harder problem — the one nobody is trying to solve. *Who do you travel with?* Solo travellers get nothing. Couples get suggestions for hotels. Anyone who wants to share a trip with someone compatible falls back to hostel bars and dating apps.

That's the gap we sat in. Not "better recommendations" — but the absence of a system that knows your rhythm, plans around it, and finds people who match it.

Advance.

Slide 3 — What Sonder is (~80 seconds)

Sonder is the only place where three things happen together.

One — real-venue itineraries. Not generic blocks. Not "explore the old town." Specific restaurants and specific shops and specific cafés at specific times on specific days, grounded in actual places that exist. We pulled the activity corpus from Foursquare's Places API across our destination universe; the LLM never invents a venue out of thin air.

Two — compatibility matching that takes the actual signal seriously. Not interest tags. Twelve push-pull motivation dimensions from the academic travel-motivation literature, an eight-key emotional signature derived from a GoEmotions cosine classifier, a multi-feature ranking engine that adjusts to your feedback after every revision.

Three — a believable social layer that's alive on day one. A hundred and ninety-two LLM-designed traveller personas, plus eighteen couples, autonomously posting, opening trips, and reaching out via chat. Cold-start solved at seed time so the first user never sees a ghost town.

The aim is simple. It should feel like a friend's recommendation. Not a search results page.

Advance.

Slide 4 — Architecture overview, first half (~60 seconds)

This slide is split with Mushahid. Cover the framing, then hand off.

At the highest level, Sonder is four service planes, each operating independently.

Cloudflare Pages serves the React frontend at the edge — three hundred plus points of presence globally. A catch-all Pages function proxies every `/api/*` request to our backend, which means the SPA stays on a single domain — required for service-worker push registration.

The backend is FastAPI on Render. Pinecone holds the vector index — fifty thousand vectors across four namespaces, with co-travellers indexed alongside hotels, restaurants, and activities. Firestore holds operational state. Firebase Storage holds the persona avatars and cached voice MP3s.

The reason these are separate planes is operational independence — Render can restart and our search vectors stay live; Pinecone can be slow and the dashboard still loads from Firestore.

Hand to Mushahid. I'll let Mushahid take you through the backend and Firestore layout.

Slide 8 — RAG itinerary generation (~90 seconds)

Ali has just walked through Pinecone and routing. You pick up the generation pipeline.

When the user hits "plan my trip," seven things happen in sequence.

Three persona-inference pipelines run in parallel — one assigns the push-pull motivation profile, one derives the emotional signature from the GoEmotions classifier, one builds the travel-style embedding. They run blind to each other on purpose.

Destination retrieval against the Pinecone destinations namespace, then activity retrieval at the per-day level, both feeding the ranking engine. The ranker scores candidates across ten features and combines them with weights the user has implicitly shaped through prior trip revisions.

Then the large-tier LLM — Claude Sonnet 4.6, with GPT-4o as fallback — streams the itinerary. The frontend reads each closing JSON brace and displays Day 1 to the user *while Day 7 is still being written*. End-to-end latency to first day is under fifteen seconds.

The validator critic runs on the finished itinerary. If it flags issues, a refinement loop regenerates against the feedback. The user never sees the validator unless something failed — when it succeeds, the itinerary just lands.

The principle running through this entire pipeline is what we call **information starvation** as architectural quality control. The persona inferrer never sees the embedding vector. The validator never sees the user's original prompt. The matcher's policy file never sees individual user data. Each stage knows exactly what it needs to do its job, and nothing else.

Advance.

Slide 10 — Validator engine (~90 seconds)

This is the part of the system I'm most proud of.

Every LLM call in Sonder is graded by a second LLM, from a different provider family, against a surface-specific rubric. We have five rubrics — one each for itinerary, persona reveal, co-traveller match, chat reply, and chat-reply repair.

The chat-reply validator alone watches ten categories. Assistant voice — is the persona slipping back into "as an AI" leakage. Semantic drift, token stutter, repetition. Romantic-vibe detection so a friendly chat doesn't turn flirty without consent. Taxonomy leakage — making sure the persona doesn't say *"I'm a story collector"* out loud, because that's internal metadata. Contradiction against the established chat memory. And several more.

A deterministic regex-based pre-check runs first — instant, no LLM cost. It catches cheap failures before they ever reach the model and reserves model spend for the genuinely ambiguous cases.

The piece I want to highlight is **async edit-in-place repair**. When the user sends a message, the persona's reply broadcasts immediately. The validator runs asynchronously in the background. If it catches something — say, a Paris persona accidentally mentioning Lisbon on a Japan trip — it writes a corrected reply, and we swap the text in place via a `message_edited` WebSocket event. The user reads the original for half a second, then sees the corrected version slide in. Large-model-validated quality at small-model latency.

The reason a Paris persona never opens with *"I see you're also interested in Lisbon?"* on a Japan trip is a three-block guardrail in every persona prompt. Hard trip scope at the top — the persona is told exactly which trip context they're in, and forbidden from referencing anything outside it. Then push-pull motivation and emotional signature as **private framing** — the persona is told these things about themselves, but instructed never to refer to them directly.

Advance.

Slide 13 — Shared-itinerary negotiation (~85 seconds)

Once two travellers have matched and both want to share a trip, we move them off the regular itinerary surface and onto a shared one. Both users edit the same document — but not directly.

Direct simultaneous edits would race. So every change goes through a **proposal-counter-accept loop**. I propose adding a sushi place on Day 3. You either accept it, or counter with a different proposal — "what about the omakase across the river instead." There is no hard reject. The negotiation must move forward; the persona evaluator is forbidden from stonewalling.

Every counter carries a brief conversational reason — *"hakone feels far after the museum day."* No naked rejections; the negotiation reads like a chat.

The doc itself uses optimistic locking on a version field. If two writes hit the server simultaneously, the second gets a 409 conflict; the client re-fetches and the user retries with the latest state. We never silently

overwrite anyone's change.

Token-Jaccard dedupe prevents the persona from circulating the same idea twice. If the LLM tries to propose the same thing in different words, the verdict flips to accept rather than looping forever.

The result is that a trip plan emerges through what reads like a real conversation, not a Google Doc with two cursors fighting.

Advance.

Slide 14 — Synthetic travellers (~80 seconds)

A travel app with no users feels dead. We solved cold-start at seed time.

A hundred and ninety-two LLM-designed traveller personas, plus eighteen couples, live in our matching pool from day one. They post on Pulse, open trips on Discover, and reach out to real users via chat — autonomously, on a background loop that runs forever.

The interesting bit is how they're written. We use a **two-stage blind-writer pipeline**.

Stage one receives only the raw signal — city, age, gender, and four persona-question option keys. It writes the character — name, voice, quirks, what they're into, how they talk. Stage two is the inferrer — it reads stage one's character output and assigns the push-pull motivation dimensions and emotional signature, blind to the original question keys.

The reason we do it this way: *a language model with the answer key in front of it writes to that key*. It produces a character that reads as "the person who'd answer this." That's not what we want. We want a character that's internally consistent first, and *then* gets classified — the same way real users do.

A language model that has to guess writes something honest. A language model that knows the answer writes something tautological.

The diversity matrix is locked at seed time — sixteen cities, three age buckets, two genders, two per cell. We backfill metadata patches through scripts, never re-paying the LLM cost — a pattern we now reuse for every schema-only field addition.

Advance.

Slide 16 — Group-style filtering (~70 seconds)

Sonder treats four user types as four genuinely different products. Solo, couple, family, friends. Trying to make one ranker serve all four would compromise every one.

Per-group itinerary planning rules. Solo gets counter-seating venues — bars where you'll actually meet people — and walking-distance stops. Couples get every overnight private, no dorms, romance-coded venues by default. Families get kids-friendly menus and dinner-by-seven. Friends get split-and-rejoin activities and apartment-over-rooms accommodation.

And — this is important — family and friends matching is **disabled**. Not de-prioritised. Off entirely.

They already have their travelling party. Surfacing strangers to a family or a friend group as "companions" makes no product sense; it's a category error. So the matching pipeline short-circuits for those groups and we don't show them mixed personas.

Solo and couples both run the matching engine, but with different filters. Solo travellers get a hard same-gender filter for safety. Couples are gender-locked at the seed level — we only seeded male+female couple pairs.

Each rule shows up as a single check in code, but the philosophy is: don't pretend they're the same product.

Advance.

Slide 19 — Closing (~25 seconds)

Final slide. Land it. Don't rush.

Sonder.

Beat.

Travel, together.

Beat.

The repo is on screen. We're open to questions.

Stop talking. Wait.

Speaker hand-offs

Memorise these. Don't search for them mid-deck.

- **Mid slide 4** — "I'll let Mushahid take you through the backend and Firestore layout."
 - **End of slide 4** — Mushahid hands back, Jahnvi takes slide 5.
 - **End of slide 7** — Ali hands to you for slide 8.
 - **End of slide 8** — "Jahnvi will take it from here on slide 9."
 - **End of slide 9** — Jahnvi hands back to you for slide 10.
 - **End of slide 10** — "Jahnvi on slide 11."
 - **End of slide 12** — Mushahid hands back to you for slide 13.
 - **End of slide 14** — "Jahnvi on slide 15."
 - **End of slide 15** — Jahnvi hands back to you for slide 16.
 - **End of slide 16** — "Mushahid on slides 17 and 18."
 - **End of slide 18** — Mushahid hands to you for the close.
-

Delivery notes

- **Slow down on slide 8 and slide 10.** They're the most technically dense moments and the audience will be wanting to catch up. Pause between concepts.
- **Slide 14 is the strongest standalone bit.** The "answer-key" line lands hard. Let it sit before moving on.
- **The "information starvation" framing on slide 8** is the philosophical anchor of the whole talk. Say it like you mean it; this is the line technical reviewers will quote back.
- **Don't apologise for anything.** Sonder is a real working system. Speak about it like the founder of a real working system would.